Implementación de pruebas y mejora del software

Breve descripción:

Este componente desarrolla habilidades para ejecutar, documentar y analizar procesos de evaluación técnica en el entorno de desarrollo desde una perspectiva de aseguramiento de calidad. Promueve el pensamiento lógico, la gestión metódica de incidentes y la formulación de planes de mejora alineados con los estándares de la industria tecnológica y las necesidades del usuario final.

Octubre 2026

Tabla de contenido

Introducción 3

1. Implementación de pruebas manuales y unitarias de software 6

1.1. Pruebas estáticas y prevención temprana 7

1.2 Pruebas de caja negra y caja blanca 9

1.3 Pruebas unitarias y aseguramiento desde el código 14

2. Gestión de resultados y evidencia de ejecución 17

2.1. Registro de hallazgos, trazabilidad y evidencias 18

2.2 Cobertura de pruebas, métricas e impacto 21

3. Gestión de incidentes y documentación de pruebas 24

3.1 Reporte de incidentes, estructura y ciclo de vida del defecto 25

3.2 Informes de resumen y métricas de calidad 31

4. Planes de mejora y optimización continua 34

4.1 Lecciones aprendidas, retrospectiva y socialización del conocimiento 35

4.2 Acciones correctivas, preventivas y estrategias de optimización 38

Síntesis 41

Glosario 43

Referencias bibliográficas 46

Créditos 47

Introducción

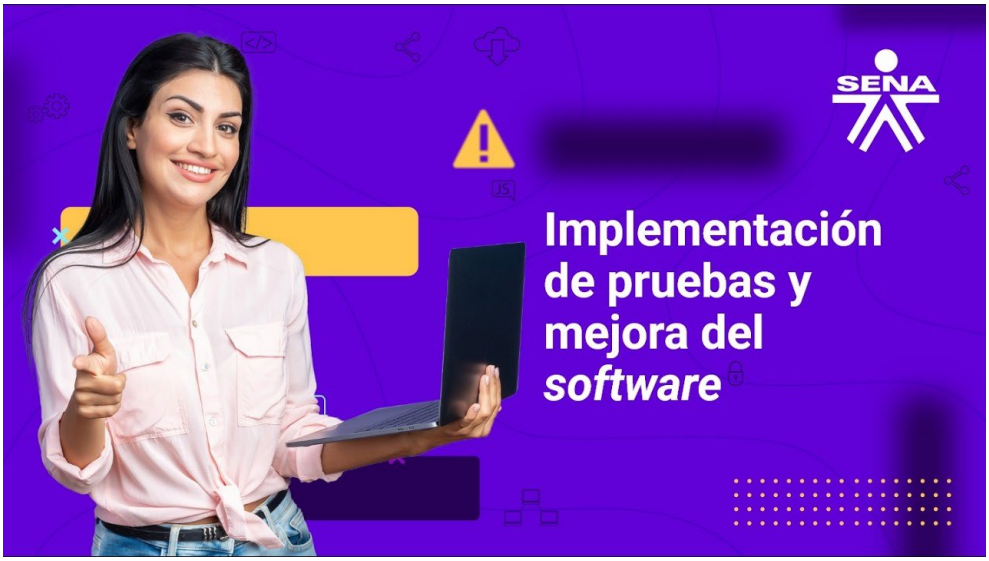
El desarrollo de software exige procesos de evaluación rigurosos que permitan identificar errores, validar funcionalidades y fortalecer la calidad de los productos tecnológicos. En este contexto, el aseguramiento de la calidad, conocido como Quality Assurance o QA, se convierte en una práctica fundamental para verificar que las soluciones respondan a los requisitos definidos, funcionen de manera confiable y ofrezcan una experiencia adecuada a las personas usuarias. Este componente formativo aborda la implementación de pruebas manuales y unitarias de software, así como la gestión de resultados y la formulación de acciones de mejora a partir de los hallazgos obtenidos.

La apropiación de estos saberes permite al aprendiz fortalecer competencias técnicas y analíticas requeridas en la industria tecnológica actual. A través del componente, se desarrollan temas relacionados con las pruebas estáticas, las pruebas de caja negra, las pruebas de caja blanca y las pruebas unitarias, comprendiendo su aplicación en diferentes momentos del proceso de desarrollo. Además, se trabaja la documentación de hallazgos, la trazabilidad de casos de prueba, la medición de cobertura, el reporte de incidentes y la consolidación de evidencias que facilitan la toma de decisiones sobre la calidad del software.

El abordaje de los contenidos integra fundamentos técnicos, ejemplos contextualizados y recursos educativos virtuales orientados a facilitar la comprensión progresiva de los temas. De esta manera, el aprendiz podrá reconocer que cada prueba ejecutada, cada defecto reportado y cada plan de mejora formulado contribuyen a entregar soluciones tecnológicas más confiables, seguras y alineadas con las

necesidades del usuario final. Este enfoque fortalece la mejora continua del proceso de desarrollo y promueve una cultura de calidad en proyectos de software.

Video 1. Implementación de pruebas y mejora del software



Enlace de reproducción del video

Transcripción del video. Implementación de pruebas y mejora del software
En el sector tecnológico colombiano, la calidad del software es esencial para que empresas, instituciones y servicios digitales entreguen soluciones confiables. La implementación de pruebas permite revisar el comportamiento del sistema, detectar defectos y generar evidencias que respalden decisiones técnicas antes de afectar a las personas usuarias. Para el aprendiz, dominar este proceso fortalece su perfil laboral en áreas de desarrollo, soporte, análisis de calidad y gestión de proyectos tecnológicos. Saber ejecutar pruebas, documentar hallazgos y proponer mejoras aumenta su empleabilidad, porque las organizaciones requieren talento capaz de reducir reprocesos, riesgos y fallas operativas.

En este componente se abordan pruebas estáticas, de caja negra, de caja blanca y unitarias; además, registro de hallazgos, trazabilidad, evidencias, cobertura, métricas, incidentes, informes y planes de mejora. Estos contenidos se articulan con los resultados de aprendizaje relacionados con implementar pruebas manuales y unitarias de software, y establecer planes de mejora según los resultados del proceso de pruebas.

Al finalizar, se espera que el aprendiz comprenda cómo convertir la ejecución de pruebas en información útil para mejorar productos y procesos. Este aprendizaje contribuye a la entrega de software seguro, funcional y alineado con requisitos, fortaleciendo la calidad del servicio tecnológico y la competitividad del territorio.

1. Implementación de pruebas manuales y unitarias de software

El proceso de aseguramiento de la calidad inicia con la comprensión profunda de las diferentes técnicas de evaluación disponibles y su correcta aplicación según la fase del proyecto. La implementación de pruebas manuales y unitarias no responde a un ejercicio de improvisación, sino a un procedimiento técnico estandarizado que busca identificar discrepancias entre el comportamiento esperado de un sistema y su comportamiento real.

En este apartado se desarrollan enfoques estáticos y dinámicos que permiten a los equipos de desarrollo detectar anomalías desde las etapas más tempranas de conceptualización hasta la validación del código ya construido. Esta sección se encuentra directamente articulada con el resultado de aprendizaje orientado a la implementación de pruebas manuales y unitarias de acuerdo con estándares y procedimientos técnicos.

- **Implementación de pruebas en el desarrollo de software**
  - **Procedimiento técnico**

    La implementación de pruebas requiere criterios, pasos y técnicas definidas previamente para evitar la improvisación.
  - **Detección de discrepancias**

    Las pruebas permiten identificar diferencias entre el comportamiento esperado del sistema y su comportamiento real.
  - **Enfoques de evaluación**

    El proceso integra enfoques estáticos y dinámicos para revisar artefactos, código y funcionalidades.

- **Relación con el aprendizaje**

  La implementación de pruebas manuales y unitarias fortalece competencias técnicas necesarias para el aseguramiento de la calidad del software.

1.1. Pruebas estáticas y prevención temprana

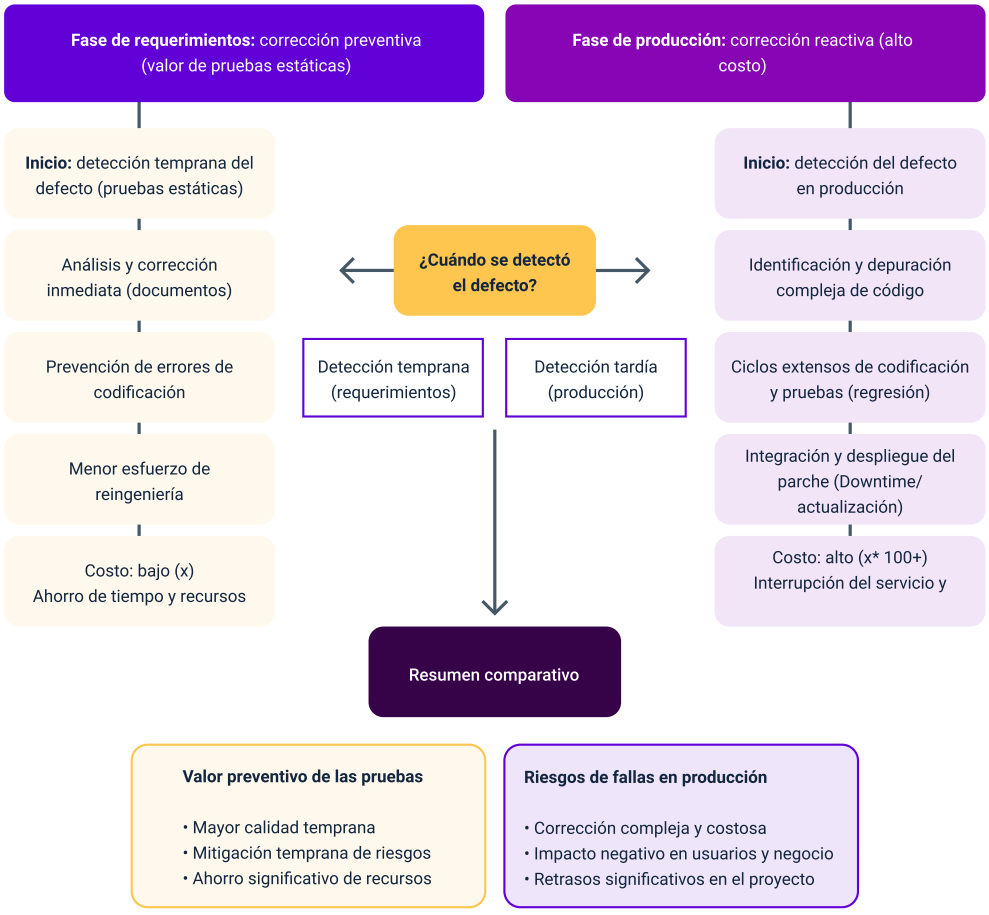
Las pruebas estáticas representan el primer filtro de calidad en cualquier proyecto de software. A diferencia de las pruebas dinámicas, que requieren la ejecución del código, las pruebas estáticas se enfocan en la revisión exhaustiva de los artefactos del proyecto antes de que el software sea compilado o ejecutado. Este enfoque preventivo se basa en la premisa de que corregir un error en la fase de requerimientos o diseño es significativamente más económico y requiere menos esfuerzo que solucionarlo cuando el sistema ya se encuentra en fase de producción.

Comprender las pruebas estáticas implica reconocerlas como un ejercicio de lectura crítica y escrutinio analítico. En este caso, no se busca “romper” el sistema interactuando con él, sino examinar la documentación, los modelos de datos, las historias de usuario y el código fuente en busca de ambigüedades, inconsistencias, desviaciones de los estándares de codificación o lógica defectuosa. Existen diversas técnicas para llevar a cabo estas pruebas, entre las que destacan las revisiones informales, las inspecciones formales, las revisiones técnicas y el análisis estático de código mediante herramientas automatizadas que detectan vulnerabilidades de seguridad o malas prácticas de programación, conocidas como code smells.

En un contexto de formación profesional, la aplicación de pruebas estáticas resulta vital para fomentar la disciplina y el trabajo colaborativo. Por ejemplo, puede pensarse en un equipo de aprendices del SENA que diseña un aplicativo web para la gestión de inventarios de una biblioteca. Antes de escribir la primera línea de código en

HTML o JavaScript, el equipo realiza una inspección formal del documento de requerimientos. Durante esta revisión, se detecta que el requerimiento “El sistema debe permitir préstamos rápidos” es ambiguo, ya que no define qué constituye “rápido” ni establece parámetros de tiempo o de interfaz. Al identificar y documentar este hallazgo mediante una prueba estática, el equipo corrige la especificación y evita desarrollar una funcionalidad que, posteriormente, no cumpliría con las expectativas del usuario final.

Figura 1. Diagrama de flujo comparativo del costo de corrección de defectos: pruebas estáticas vs. Producción



Esta práctica de revisión temprana se alinea directamente con el resultado de aprendizaje de implementar pruebas de acuerdo con procedimientos técnicos, demostrando que la calidad se construye desde la planeación. Las pruebas estáticas instauran una cultura de prevención, donde la meticulosidad en el análisis documental se convierte en la base para un desarrollo de software robusto y libre de defectos estructurales.

1.2 Pruebas de caja negra y caja blanca

Las pruebas de caja negra, también conocidas como pruebas basadas en la especificación o pruebas funcionales, evalúan el comportamiento del software desde una perspectiva externa, ignorando por completo la arquitectura o el código fuente subyacente. En este enfoque, el sistema es visto como una "caja negra", donde el evaluador proporciona datos de entrada, conocidos como inputs, y verifica que los datos de salida, conocidos como outputs, coincidan con los resultados esperados según los requerimientos del cliente.

La relevancia de este método radica en su capacidad para validar la experiencia final del usuario y asegurar que las funcionalidades del sistema operen de manera congruente con las necesidades de negocio. Desde una perspectiva instruccional, ejecutar pruebas de caja negra requiere adoptar la mentalidad del usuario final, dejando de lado los conocimientos técnicos sobre cómo está programado el sistema.

El diseño de estas pruebas implica la creación de escenarios y casos de uso basados exclusivamente en manuales, historias de usuario o especificaciones funcionales. Para optimizar este proceso y evitar la ejecución de un número excesivo de pruebas, se emplean técnicas de diseño formales como la partición de clases de equivalencia, que consiste en agrupar datos de entrada que deben generar el mismo

comportamiento, y el análisis de valores límite, que permite probar los extremos numéricos o lógicos donde es más probable que ocurran errores.

Para ilustrar este concepto en un escenario aplicable, puede considerarse el desarrollo de una plataforma de comercio electrónico para artesanos locales. El sistema posee un campo de texto para ingresar la cantidad de productos a comprar, el cual, según los requerimientos, solo debe aceptar números enteros entre 1 y 50. Aplicando la técnica de análisis de valores límite en una prueba de caja negra, no se revisa si el código usa un ciclo `if` o un `switch`; en su lugar, se ejecutan casos de prueba ingresando los valores 0, como límite inferior inválido; 1, como límite inferior válido; 50, como límite superior válido; y 51, como límite superior inválido. Si al ingresar 51 el sistema permite la compra en lugar de arrojar un mensaje de error, la prueba se registra como fallida.

Este tipo de pruebas asegura que la interfaz interactúe correctamente, que los flujos de navegación sean lógicos y que las restricciones operativas se cumplan a cabalidad, validando así el valor real que el producto tecnológico entrega a la sociedad.

- **Técnicas aplicadas en pruebas de caja negra**
  - **Perspectiva externa**

    Las pruebas de caja negra validan el comportamiento del sistema desde la experiencia del usuario, sin revisar la estructura interna del código.
  - **Inputs y outputs**

    El evaluador ingresa datos de entrada y verifica que las salidas coincidan con los resultados esperados según los requerimientos.

- **Partición de equivalente**

  Agrupa datos de entrada que deberían generar el mismo comportamiento, para reducir la cantidad de pruebas sin perder cobertura funcional.

- **Valores límites**

  Evalúa los extremos permitidos o no permitidos por el sistema, porque allí suelen presentarse errores con mayor frecuencia.

- **Ejemplo aplicado**

  En una compra con rango permitido entre 1 y 50 productos, se prueban valores como 0, 1, 50 y 51 para validar límites válidos e inválidos.

Las pruebas de caja blanca, también llamadas pruebas estructurales, de caja de cristal o de caja transparente, se enfocan en la anatomía interna del software. Este enfoque requiere conocimiento profundo y detallado del código fuente, la arquitectura de la base de datos, los algoritmos y los flujos de control de la aplicación.

El objetivo principal de las pruebas de caja blanca es verificar la estructura interna del sistema, asegurando que todas las rutas de código posibles, las condiciones lógicas y los bucles operen de manera eficiente, segura y sin errores ocultos. A diferencia de las pruebas de caja negra, aquí no solo interesa comprobar qué resultado entrega el sistema, sino cómo llega a ese resultado.

El proceso de realizar pruebas de caja blanca es altamente técnico y requiere bases sólidas en programación. Pedagógicamente, puede explicarse como la validación de la “tubería interna” de la aplicación. Mientras la caja negra verifica si sale agua limpia por el grifo, la caja blanca verifica que las tuberías no tengan fisuras, que las válvulas abran en el momento exacto y que no existan tramos inútiles, equivalentes al código muerto.

Las técnicas utilizadas en este nivel incluyen la cobertura de sentencias, que busca garantizar que cada línea de código se ejecute al menos una vez durante las pruebas; la cobertura de decisiones, que valida que las condiciones verdaderas y falsas sean evaluadas; y la cobertura de rutas, que permite probar los caminos lógicos posibles desde el inicio hasta el fin de un módulo.

Para comprender su aplicabilidad en la formación profesional, puede tomarse como ejemplo un sistema de información desarrollado por aprendices SENA que incluye un módulo de autenticación de usuarios, conocido como `login`. Al realizar pruebas de caja blanca sobre este módulo, se revisa directamente la función de validación escrita en lenguaje PHP o `Python`. Allí se analiza el bloque condicional que verifica si la contraseña coincide con el `hash` almacenado en la base de datos. También se diseña una prueba específica para forzar la ejecución de la cláusula `else` del código, es decir, el camino lógico que maneja el error de contraseña incorrecta, verificando además que no existan vulnerabilidades de inyección SQL en la consulta que interactúa con la base de datos.

- **Visualización de cobertura de rutas: pruebas de caja blanca**

  a) **Diagrama de flujo de control**

     Representa las decisiones, condiciones y caminos que puede seguir el programa durante su ejecución.

  b) **Rutas posibles**

     Muestra los caminos que se pueden recorrer según las condiciones evaluadas en el código.

c) Casos de prueba

Relaciona combinaciones de entrada con las rutas que se ejecutan durante la prueba.

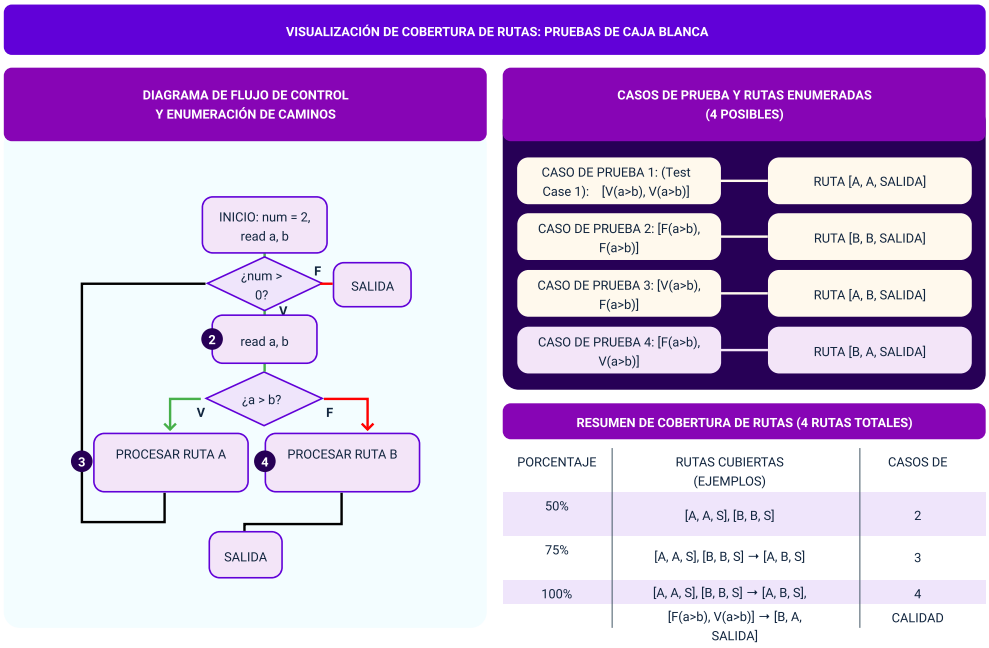
d) Porcentaje de cobertura

Permite identificar qué tanto del flujo lógico ha sido evaluado mediante los casos de prueba.

e) Calidad del código

Una mayor cobertura ayuda a detectar rutas no evaluadas, errores ocultos y debilidades en la lógica interna del software.

Figura 2. Visualización de cobertura de rutas: pruebas de caja blanca



Nota. Elaboración propia con base en los principios de pruebas de software del International Software Testing Qualifications Board (ISTQB, 2024).

La integración de las pruebas de caja blanca en el perfil del egresado satisface el resultado de aprendizaje orientado a la implementación técnica. Fomenta un nivel de

ingeniería de software más profundo, donde el desarrollador no solo se conforma con que el software funcione aparentemente bien, sino que garantiza que la lógica constructiva subyacente posea altos estándares de calidad, mantenibilidad y seguridad.

1.3 Pruebas unitarias y aseguramiento desde el código

Las pruebas unitarias constituyen el nivel más granular y fundamental de las pruebas dinámicas de software. Su propósito es aislar y evaluar la unidad de código más pequeña comprobable de una aplicación, que generalmente corresponde a una función, un método o una clase específica, dependiendo del paradigma de programación utilizado. La filosofía detrás de las pruebas unitarias es verificar que cada pieza individual del rompecabezas funcione correctamente de manera independiente, antes de intentar ensamblar el sistema completo. Este tipo de pruebas son, por lo general, diseñadas, escritas y ejecutadas por los mismos desarrolladores durante la fase de codificación.

La enseñanza de las pruebas unitarias se aborda como una práctica de higiene en el desarrollo de software. No se trata simplemente de escribir código para probar código, sino de una metodología que estructura el pensamiento lógico. El proceso implica el uso de frameworks de automatización, como JUnit para Java, PyTest para Python o Jest para JavaScript, lo que permite que estas pruebas se ejecuten de manera rápida y repetitiva. Al estar automatizadas, proporcionan una red de seguridad: si se modifica una función existente para agregar una nueva característica, las pruebas unitarias permiten detectar de inmediato si ese cambio alteró el comportamiento original de la función, es decir, si generó una regresión.

A modo de ejemplo concreto, puede pensarse en el desarrollo de una función en JavaScript llamada calcularCostoEnvio (distancia, peso). Esta pequeña unidad de código debe retornar el valor del flete con base en dos parámetros matemáticos. Para realizar la prueba unitaria, se escribe un pequeño script de prueba utilizando el entorno Jest, donde se afirma, mediante una validación o assert, que, si la distancia es 10 km y el peso es 5 kg, la función debe retornar exactamente $15.000 COP. Si el código fuente tiene un error en la fórmula matemática, la prueba unitaria fallará de manera inmediata, localizando el error con precisión en esa función específica, sin necesidad de cargar la interfaz gráfica ni consultar bases de datos externas.

Al dominar las pruebas unitarias, el futuro profesional contribuye a que los cimientos técnicos de cualquier aplicativo sean robustos, escalables y confiables.

- **Secuencia básica de una prueba unitaria**
  - **Paso 1**

    Identificar la unidad

    Se selecciona la función, método o clase que será evaluada de forma aislada.
  - **Paso 2**

    Definir datos de entrada

    Se establecen los valores que permitirán comprobar el comportamiento esperado de la unidad.
  - **Paso 3**

    Ejecutar la validación

    Se utiliza un framework de pruebas para ejecutar la función y contrastar el resultado obtenido.

- **Paso 4**

  Analizar el resultado

  Se verifica si la salida coincide con el comportamiento esperado o si existe un error que requiere corrección.

2. Gestión de resultados y evidencia de ejecución

La ejecución de pruebas, sean estáticas o dinámicas, carece de valor si sus hallazgos no se gestionan de manera sistemática. La transición entre la fase operativa de evaluación y la fase analítica de mejora marca la diferencia entre un equipo que simplemente encuentra errores y uno que asegura la calidad integral del producto.

Este apartado se orienta a establecer planes de mejora basados en la evidencia empírica obtenida durante las pruebas. Para lograrlo, el aprendiz debe dominar procesos como el registro documental, el análisis de cobertura, la gestión de evidencias y la interpretación de resultados. La calidad de un software no se mide únicamente por la ausencia de fallas en su lanzamiento, sino por la rigurosidad con la que se documentaron, analizaron y resolvieron los problemas durante su construcción.

- **De la ejecución de pruebas a la mejora**
  - **Ejecutar pruebas**

    Las pruebas permiten comparar el comportamiento esperado del sistema con el comportamiento real obtenido durante la evaluación.
  - **Registrar hallazgos**

    Cada resultado debe documentarse de manera objetiva, indicando si la prueba fue exitosa o fallida.
  - **Reunir evidencias**

    Las evidencias permiten demostrar lo ocurrido durante la ejecución y facilitan la reproducción de fallas.
  - **Analizar resultados**

    La información registrada sirve como base para medir cobertura, identificar riesgos y orientar decisiones de mejora.

2.1. Registro de hallazgos, trazabilidad y evidencias

El primer eslabón en la cadena de la gestión de calidad es la captura objetiva de los resultados de cada caso de prueba ejecutado. Este proceso exige que el profesional en aseguramiento de calidad registre, de manera detallada, si el comportamiento observado en el sistema coincidió con el comportamiento esperado.

Cuando ambas variables coinciden, el hallazgo se registra como exitoso, es decir, successful o pass. Cuando existe una desviación, por mínima que sea, se registra como fallido, es decir, fail. Esta categorización es fundamental para establecer el estado de salud del proyecto tecnológico en un momento determinado.

Desde una perspectiva pedagógica, el registro de hallazgos enseña el valor de la objetividad y la precisión. No basta con afirmar que un formulario “no funciona”; es necesario documentar en qué condiciones específicas se produjo la falla. El registro debe incluir la fecha y hora de la ejecución, el entorno de pruebas utilizado, por ejemplo, navegador, sistema operativo o versión del dispositivo móvil, los datos de entrada empleados y la salida real generada por el sistema.

Esta información constituye la materia prima sobre la cual los desarrolladores trabajarán posteriormente para aplicar las correcciones necesarias.

- **Elementos del registro de hallazgos**
  - **Estado de la prueba**

    Indica si el caso de prueba fue exitoso, successful o pass, o si fue fallido, fail, según la comparación entre el resultado esperado y el resultado obtenido.

- **Condiciones de ejecución**

  Incluye datos como fecha, hora, navegador, sistema operativo, versión del dispositivo o entorno en el que se realizó la prueba.

- **Datos de entrada**

  Registra los valores específicos utilizados durante la prueba, ya que estos ayudan a reproducir el comportamiento del sistema.

- **Salida real del sistema**

  Describe la respuesta obtenida durante la ejecución, especialmente cuando existe una diferencia frente al resultado esperado.

La trazabilidad es el principio que permite vincular cada prueba ejecutada con un requerimiento de negocio o funcional específico. Para garantizar esta conexión, en la industria tecnológica se emplea la Matriz de Trazabilidad de Requerimientos, conocida como RTM, por sus siglas en inglés. La comprensión de este instrumento es vital, ya que permite visualizar cómo una necesidad del cliente se transforma en una característica del software y, a su vez, cómo esa característica es validada por uno o varios casos de prueba.

En la práctica formativa, la trazabilidad evita que el equipo de desarrollo escriba pruebas innecesarias o que olvide evaluar funcionalidades críticas. Por ejemplo, en el desarrollo de un portal de educación virtual, un requerimiento puede estipular que “el sistema debe permitir la carga de archivos en formato PDF de hasta 5 MB”. La matriz de trazabilidad conectará este requerimiento con un caso de prueba de caja negra diseñado para validar precisamente esa restricción. Si el caso de prueba falla, la trazabilidad permite identificar inmediatamente qué requerimiento del cliente se encuentra en riesgo de incumplimiento.

Un hallazgo registrado sin evidencia carece de validez técnica. La estructuración de la evidencia implica recolectar artefactos digitales que demuestren de forma irrefutable el resultado de la prueba. Para los casos exitosos, la evidencia asegura que la funcionalidad fue validada correctamente antes de la entrega; para los casos fallidos, la evidencia es crucial para que el desarrollador pueda reproducir y comprender el error.

Las formas más comunes de estructurar esta evidencia incluyen capturas de pantalla con anotaciones, grabaciones de video de la interacción con la interfaz, archivos de registro o logs generados por el servidor, y exportaciones del estado de la base de datos antes y después de la transacción. Pedagógicamente, la evidencia debe entenderse como un instrumento de comunicación técnica, no como una herramienta para penalizar al programador. Una evidencia bien estructurada elimina la ambigüedad, reduce los tiempos de depuración y fomenta un ambiente de trabajo colaborativo fundamentado en datos objetivos.

- **Ruta para documentar hallazgos con evidencia**
  - **Paso 1**

    Registrar el resultado

    Se indica si el caso de prueba fue exitoso o fallido, comparando el comportamiento esperado con el comportamiento real.
  - **Paso 2**

    Relacionar el requerimiento

    Se vincula el caso de prueba con el requerimiento correspondiente mediante una matriz de trazabilidad o RTM.

- **Paso 3**

  Recolectar evidencias

  Se guardan capturas, grabaciones, logs o exportaciones que permitan demostrar lo ocurrido durante la ejecución.

- **Paso 4**

  Facilitar la corrección

  La evidencia se organiza para que el equipo de desarrollo pueda reproducir, analizar y corregir la falla con mayor precisión.

2.2 Cobertura de pruebas, métricas e impacto

Determinar cuándo se ha probado lo suficiente es uno de los dilemas más recurrentes en el desarrollo de software. Es matemáticamente imposible ejecutar un número infinito de pruebas para cubrir todas las combinaciones posibles de datos e interacciones en un sistema complejo. Frente a esta limitación, surge el concepto de cobertura de pruebas, el cual permite cuantificar el alcance y la profundidad del esfuerzo de evaluación realizado.

Cuantificar la cobertura es esencial para tomar decisiones informadas sobre la liberación de un producto al mercado o su paso a la siguiente fase de desarrollo.

Las métricas de cobertura proporcionan un indicador numérico que refleja el progreso de la fase de aseguramiento de calidad. Existen diferentes niveles para medir esta cobertura. Por un lado, la cobertura de requerimientos calcula el porcentaje de funcionalidades solicitadas por el cliente que ya cuentan con casos de prueba diseñados y ejecutados. Por otro lado, la cobertura de código, más ligada a las pruebas de caja blanca y unitarias, calcula el porcentaje de líneas de código fuente que han sido ejecutadas por las pruebas automáticas.

Para ilustrar este cálculo de manera sencilla en un contexto formativo, la fórmula básica establece que la cobertura es igual al número de elementos evaluados dividido por el número total de elementos, multiplicado por cien. Si un aplicativo de gestión documental tiene 50 requerimientos funcionales aprobados y el aprendiz ha ejecutado pruebas exitosas para 40 de ellos, la cobertura funcional actual es del 80 %. Comprender estas métricas permite evaluar el riesgo residual del proyecto, asimilando que un nivel bajo de cobertura implica una alta probabilidad de que existan defectos ocultos en las áreas no evaluadas.

- **Fórmula básica de cobertura**

  Cobertura de pruebas = elementos evaluados / total de elementos × 100

  Ejemplo: si existen 50 requerimientos funcionales y se han ejecutado pruebas exitosas para 40 de ellos, la cobertura funcional es del 80 %.

El análisis de impacto es el proceso analítico que sigue a la cuantificación matemática de la cobertura. No todas las funcionalidades de un software poseen el mismo nivel de criticidad. Por tanto, una cobertura del 90 % podría parecer excelente, pero si el 10 % restante corresponde al módulo de pagos electrónicos de una tienda virtual, el riesgo para el negocio es inaceptablemente alto.

Con este análisis, la cobertura deja de interpretarse solo como un porcentaje y empieza a leerse en relación con la importancia estratégica de cada componente. Por esta razón, no basta con afirmar que se ha cubierto una alta proporción del sistema; también es necesario identificar qué funcionalidades quedaron sin evaluar, qué nivel de riesgo representan y qué consecuencias podrían generar para el usuario, el negocio o la operación del sistema.

El aprendiz debe aprender a ponderar las métricas de cobertura con base en la importancia estratégica de los componentes del sistema. Este análisis de completitud orienta los esfuerzos del equipo hacia las áreas de mayor riesgo. Si durante la cuantificación se identifica que un módulo crítico carece de casos de prueba, el líder de calidad debe ordenar el diseño y ejecución inmediata de evaluaciones para esa sección.

Así, la cuantificación deja de ser un simple reporte estadístico y se convierte en una herramienta dinámica de gestión de riesgos, permitiendo al profesional en formación tomar decisiones estratégicas sobre dónde invertir el tiempo y los recursos de pruebas restantes.

- **Decisiones a partir de la cobertura**
  - **Medir cobertura**

    Se calcula qué proporción de requerimientos, funcionalidades o código ha sido evaluada mediante casos de prueba.
  - **Identificar vacíos**

    Se reconocen áreas del sistema que aún no han sido evaluadas o que presentan baja cobertura.
  - **Analizar criticidad**

    Se determina si los componentes no cubiertos son de bajo, medio o alto impacto para el negocio o el usuario.
  - **Priorizar nuevas pruebas**

    Se decide dónde invertir tiempo y recursos para reducir riesgos antes de liberar o avanzar el producto.

3. Gestión de incidentes y documentación de pruebas

La gestión de incidentes y la documentación de pruebas permiten transformar los resultados de la ejecución en información útil para el equipo de desarrollo, los líderes del proyecto y las partes interesadas. Cuando una prueba falla, no basta con indicar que existe un error; es necesario registrar el incidente con precisión, explicar cómo reproducirlo, documentar la evidencia y consolidar los resultados para facilitar la toma de decisiones sobre la calidad del software.

Este proceso fortalece la comunicación técnica entre quienes evalúan el sistema y quienes realizan las correcciones. Además, permite identificar patrones de fallas, analizar la eficiencia del proceso de pruebas y presentar una visión clara del estado real del producto tecnológico.

- **De la falla al informe de calidad**

  a) **Prueba fallida**

     La prueba evidencia una diferencia entre el resultado esperado y el resultado obtenido.

  b) **Reporte de incidente**

     El hallazgo se documenta con precisión técnica para facilitar su análisis y corrección.

  c) **Gestión del defecto**

     El incidente pasa por estados de revisión, asignación, corrección, confirmación y cierre.

  d) **Documentación integral**

     Los casos, métricas e incidentes se consolidan en informes que apoyan la toma de decisiones.

3.1 Reporte de incidentes, estructura y ciclo de vida del defecto

Cuando una prueba se registra como fallida, el proceso natural es la creación de un reporte de incidente, comúnmente conocido en el ámbito informático como bug. Reportar un incidente no consiste simplemente en notificar que algo se ha roto; es un proceso formal de documentación que exige precisión técnica y claridad descriptiva.

La calidad del reporte de un defecto impacta directamente en la velocidad con la que el equipo de desarrollo podrá solucionarlo. Por esta razón, esta competencia se convierte en un pilar importante dentro del aseguramiento de la calidad, ya que permite comunicar el problema de forma comprensible, reproducible y útil para su corrección.

Antes de continuar con el ciclo de vida del defecto, se presenta el pódcast "Cuando una prueba falla: reportar, corregir y aprender". Este recurso permite comprender, mediante una conversación aplicada, por qué una prueba fallida debe convertirse en información técnica clara, completa y reproducible. A través de un ejemplo relacionado con la carga de archivos PDF en una plataforma académica, se explica la importancia de documentar el incidente, registrar evidencias, diferenciar el resultado esperado del resultado real y facilitar la comunicación entre el evaluador, el desarrollador y el líder técnico.

Transcripción del pódcast. Cuando una prueba falla: reportar, corregir y aprender
Persona 1: bienvenidos a todos, hoy nos convoca un tema vital para el desarrollo de pruebas en software testing. En este episodio hablaremos sobre el reporte de incidentes y el ciclo de vida del defecto, dos elementos fundamentales para garantizar la calidad del software.

Persona 2: así es. Para empezar, piensa en una plataforma académica que debe permitir cargar evidencias en PDF. Durante la prueba, el archivo cumple con el tamaño permitido, pero el sistema muestra error y no lo recibe. El defecto existe, pero el reporte queda incompleto. ¿Qué pasa cuando una prueba falla y el incidente no se documenta bien?

Persona 1: el error puede perderse, repetirse o llegar hasta producción. En pruebas de software, detectar una falla no es suficiente. El incidente debe convertirse en información técnica clara. Un buen reporte explica qué ocurrió, dónde ocurrió, con qué datos se ejecutó la prueba y cuál fue la diferencia entre el resultado esperado y el resultado real.

Persona 2: entonces, ¿el reporte del incidente funciona como una guía para que el equipo pueda corregir el problema?

Persona 1: ¡exactamente! El reporte resulta útil por tres razones principales. Primero, permite reproducir la falla mediante pasos claros y evidencias. Segundo, ayuda a priorizar el trabajo porque identifica la severidad y la urgencia del defecto. Y tercero, organiza la comunicación entre el evaluador o tester, y el desarrollador y el líder técnico, evitando interpretaciones confusas o correcciones incompletas.

Persona 2: ¿y cómo se vería eso en un contexto productivo cercano al aprendiz?

Persona 1: pues imagínese un portal de formación donde el requisito indica que se pueden cargar archivos PDF de hasta 5 megabytes. Si un archivo de 4 megabytes falla, el reporte debe incluir un título descriptivo, navegador utilizado, versión del sistema, pasos realizados, resultado esperado, resultado actual y evidencia, como una captura o registro. Con esa información, el desarrollador

puede ubicar la causa, corregirla y devolver el caso para una prueba de confirmación.

Persona 2: entonces, ¿qué puede salir bien cuando se gestiona correctamente el ciclo de vida del defecto y qué puede salir mal si no se controla?

Persona 1: cuando se gestiona bien, el defecto pasa por estados como nuevo, asignado, en progreso, resuelto, cerrado o reabierto. Esto permite hacer seguimiento y comprobar que la corrección realmente funcionó. Cuando no se controla, el incidente puede quedar sin responsable, corregirse solo parcialmente, repetirse en otro ciclo o afectar la confianza del usuario final.

Persona 2: en pocas palabras, una prueba fallida no termina en el error. Empieza un proceso de comunicación, corrección y aprendizaje.

Persona 1: así es. Por eso, al estudiar este tema, se recomienda revisar el punto 3.1 del componente formativo dedicado al reporte de incidentes, su estructura y el ciclo de vida del defecto. Allí se explica cómo transformar una falla detectada en información útil para mejorar la calidad del software y fortalecer el trabajo del equipo de desarrollo.

Persona 2: porque en aseguramiento de la calidad, detectar una falla es importante. Pero convertir esa falla en información útil para la mejora continua es lo que realmente genera valor.

Persona 1: gracias por acompañarnos, nos escuchamos en el próximo episodio.

El concepto central en la gestión de incidentes es el ciclo de vida del defecto, el cual describe los diferentes estados por los que atraviesa un error desde el momento en que es descubierto hasta que es solucionado y cerrado definitivamente. En un

entorno profesional o formativo simulado, este flujo de trabajo organiza la comunicación entre el evaluador, también llamado `tester`, y el programador o desarrollador.

Un ciclo de vida estándar inicia cuando el incidente es reportado con el estado “Nuevo”. Posteriormente, un líder técnico lo revisa y, si es válido, lo pasa al estado “Asignado”, derivándolo al desarrollador responsable. Mientras el programador trabaja en la solución, el estado cambia a “En progreso”. Una vez corregido el código, el defecto se marca como “Resuelto” o “Listo para prueba”.

En este punto, la responsabilidad retorna al evaluador, quien debe ejecutar nuevamente la prueba original, conocida como prueba de confirmación. Si el error desapareció, el defecto se marca como “Cerrado”; si persiste, se marca como “Reabierto”, reiniciando el ciclo. Este flujo permite comprender que la detección de un error es solo el inicio de un proceso colaborativo de saneamiento del código.

- **Ciclo de vida del defecto**
  - **Momento 1**

    Nuevo

    El incidente se reporta por primera vez después de identificar una diferencia entre el resultado esperado y el resultado obtenido.
  - **Momento 2**

    Asignado

    Un líder técnico revisa el incidente y lo deriva al desarrollador responsable cuando el reporte es válido.

- **Momento 3**

  En progreso

  El desarrollador analiza la causa del defecto y trabaja en la corrección correspondiente.

- **Momento 4**

  Resuelto o listo para prueba

  El código ha sido corregido y el defecto queda disponible para una nueva validación.

- **Momento 4**

  Cerrado o reabierto

  Si la prueba de confirmación demuestra que el error fue solucionado, el defecto se cierra. Si el error persiste, se reabre y el ciclo continúa.

Para que el ciclo de vida fluya sin fricciones, el documento o `ticket` que describe el incidente debe estar estructurado meticulosamente. Un reporte ineficiente, que incluya frases vagas como “la pantalla se queda en blanco”, genera frustración y pérdida de tiempo. Por el contrario, un reporte bien elaborado actúa como un manual de instrucciones para reproducir la falla.

Todo incidente debe contener un título descriptivo y conciso, la severidad del error, la prioridad, el entorno de ejecución donde ocurrió la falla y, especialmente, los pasos para reproducir el problema. Además, se debe contrastar de manera explícita el resultado esperado frente al resultado actual, adjuntando la evidencia estructurada previamente. Al dominar esta estructura, se facilita la resolución ágil de problemas y se demuestra profesionalismo en la comunicación técnica.

- **Estructura de un reporte de error efectivo**
  - **Título descriptivo**

    Resume el defecto de manera clara y concisa para que el equipo identifique rápidamente el problema reportado.
  - **Severidad**

    Indica el impacto del error en el sistema, desde un defecto crítico hasta uno cosmético.
  - **Prioridad**

    Define la urgencia con la que el defecto debe ser atendido por el equipo responsable.
  - **Entorno de ejecución**

    Describe las condiciones en las que ocurrió la falla, como navegador, sistema operativo, versión o dispositivo utilizado.
  - **Pasos para reproducir**

    Presenta la secuencia exacta de acciones realizadas por el evaluador para que el equipo pueda repetir la falla.
  - **Resultado esperado y resultado actual**

    Compara lo que el sistema debía hacer con lo que realmente ocurrió durante la ejecución.
  - **Evidencia asociada**

    Incluye capturas, grabaciones, archivos de registro o documentos que permitan comprender y validar el incidente.

3.2 Informes de resumen y métricas de calidad

A medida que finaliza un ciclo o nivel de pruebas, toda la información fragmentada, como casos exitosos, casos fallidos, métricas de cobertura y reportes de incidentes, debe ser consolidada para ofrecer una visión panorámica de la calidad del producto. La documentación integral es el mecanismo a través del cual el equipo de calidad comunica el estado real del software a las partes interesadas, conocidas como stakeholders, tales como gerentes de proyecto, clientes o, en el contexto formativo, instructores.

El informe de resumen de pruebas es un documento formal que compila los resultados de todo el esfuerzo de aseguramiento de calidad. Este artefacto no debe ser una lista interminable de datos técnicos, sino un resumen ejecutivo diseñado para facilitar la toma de decisiones.

En este informe se debe sintetizar qué componentes del sistema fueron evaluados, qué técnicas se aplicaron, qué entorno se configuró y cuáles fueron los principales obstáculos encontrados. La redacción de este informe fomenta la capacidad de síntesis y la comunicación gerencial.

Por ejemplo, en lugar de detallar cada uno de los 200 incidentes encontrados en un sistema de reservas en línea, el informe puede agrupar la información destacando que el 60 % de las fallas se concentraron en la pasarela de pagos, lo cual señala un problema sistémico en la integración con el banco. Esta consolidación permite a los líderes del proyecto entender dónde están los cuellos de botella de calidad sin perderse en el microdetalle de los casos individuales.

- **Contenido de un informe de resumen de pruebas**
  - **Componente evaluación**

    Identifica los módulos, funcionalidades o documentos que hicieron parte del ciclo de pruebas.
  - **Técnicas aplicadas**

    Resume los tipos de pruebas utilizados, como pruebas funcionales, no funcionales, estáticas, dinámicas, de caja negra o de caja blanca.
  - **Entorno configurado**

    Describe las condiciones técnicas utilizadas durante la ejecución, como ambiente, navegador, sistema operativo, versión o dispositivos.
  - **Obstáculos encontrados**

    Presenta las principales dificultades del proceso, tales como fallas recurrentes, bloqueos, limitaciones de entorno o problemas de integración.
  - **Resultados relevantes**

    Sintetiza los hallazgos más importantes para apoyar decisiones sobre corrección, liberación o continuidad del proyecto.

Junto con el resumen narrativo, la documentación integral debe incluir una sección cuantitativa robusta. La consolidación de métricas de calidad agrupa indicadores clave de rendimiento, conocidos como KPIs, relacionados con el proceso de pruebas.

Algunas de las métricas más relevantes incluyen la densidad de defectos, entendida como el número de errores encontrados por cada mil líneas de código o por módulo funcional; el tiempo promedio de resolución de incidentes; y la tasa de

defectos rechazados, es decir, reportes creados por el tester que resultaron no ser errores reales.

El análisis de estas métricas permite evaluar no solo la calidad del software, sino también la eficiencia del propio proceso de pruebas. Si la tasa de defectos rechazados es muy alta, puede sugerir que el equipo de pruebas no comprende adecuadamente los requerimientos funcionales. Si el tiempo de resolución es excesivamente largo, puede indicar falta de recursos o de pericia técnica en el equipo de desarrollo.

Aprender a interpretar estos datos dota al aprendiz de herramientas para liderar procesos de calidad basados en evidencia empírica.

- **Métricas para evaluar la calidad del proceso**
  - **Densidad de defectos**

    Relaciona la cantidad de errores encontrados con el tamaño del código o con un módulo funcional específico.
  - **Tiempo promedio de resolución**

    Permite estimar cuánto tarda el equipo en solucionar los incidentes reportados.
  - **Tasa de defectos rechazados**

    Indica cuántos reportes creados por el tester no correspondían realmente a defectos del sistema.
  - **Interpretación de resultados**

    Ayuda a identificar problemas de comprensión de requisitos, falta de recursos o necesidades de mejora en el proceso de pruebas.

4. Planes de mejora y optimización continua

La gestión de resultados no finaliza con el registro de hallazgos, incidentes o métricas. Para que el proceso de pruebas aporte valor real al desarrollo de software, la información recopilada debe transformarse en acciones de mejora. Esta fase permite analizar lo ocurrido durante la ejecución, reconocer fallas recurrentes, socializar aprendizajes y formular estrategias que fortalezcan tanto el producto tecnológico como el proceso de trabajo del equipo.

Los planes de mejora permiten pasar de la identificación del problema a la definición de acciones concretas, asignables y medibles. De esta manera, el proceso de testing no se entiende como un obstáculo dentro del desarrollo, sino como una fuente de aprendizaje técnico y organizacional que impulsa la calidad, la eficiencia y la evolución continua del software.

- **De los resultados a la mejora continua**
  - **Analizar resultados**

    Los registros, métricas e incidentes permiten identificar qué ocurrió durante el proceso de pruebas.
  - **Reconocer causas**

    El análisis permite diferenciar errores del código, fallas de comunicación, retrasos o problemas del proceso.
  - **Socializar aprendizajes**

    Las lecciones aprendidas deben compartirse para fortalecer el conocimiento del equipo de desarrollo.

- **Formular mejoras**

  Los hallazgos se convierten en acciones correctivas, preventivas y estrategias de optimización continua.

4.1 Lecciones aprendidas, retrospectiva y socialización del conocimiento

El ciclo de desarrollo y pruebas de software es, por naturaleza, iterativo y empírico. Una vez finalizada la documentación formal de los resultados, el equipo debe llevar a cabo un ejercicio de introspección para evaluar su propio desempeño. Este análisis, a menudo enmarcado en ceremonias ágiles como la retrospectiva, es crucial para cultivar una mentalidad de mejora continua.

Identificar qué funcionó bien y qué falló en el proceso de evaluación asegura que los errores metodológicos no se repitan en futuras iteraciones o proyectos. Por esta razón, las lecciones aprendidas no deben asumirse como un cierre administrativo, sino como una oportunidad para mejorar la forma en que el equipo diseña, ejecuta, documenta y analiza las pruebas.

El primer paso de la retrospectiva consiste en realizar un análisis de causa raíz sobre las fallas procedimentales. Es fundamental diferenciar esto de los errores del código; aquí se evalúan los errores del equipo humano. Por ejemplo, si se descubre un defecto crítico de caja negra un día antes del lanzamiento oficial, es necesario analizar si el diseño de los casos de prueba comenzó demasiado tarde o si el entorno de pruebas no estaba configurado a tiempo.

Mediante técnicas de análisis como los “5 Porqués” o el Diagrama de Ishikawa, el aprendiz aprende a profundizar más allá de los síntomas superficiales para encontrar el origen de los problemas organizacionales. Por ejemplo, si un equipo detecta reiteradamente que sus casos de prueba están desactualizados frente a la versión final

del software, la lección aprendida podría indicar una falla de comunicación entre los analistas de requerimientos y el equipo de aseguramiento de calidad.

Identificar estas oportunidades de mejora es un componente central de la madurez profesional y del cumplimiento del resultado de aprendizaje asociado a la formulación de planes de mejora.

- **Ruta para identificar lecciones aprendidas**
  - **Paso 1**

    Revisar el proceso

    Se analiza cómo se diseñaron, ejecutaron y documentaron las pruebas durante el ciclo de trabajo.
  - **Paso 2**

    Identificar fallas recurrentes

    Se reconocen problemas repetidos, como casos de prueba desactualizados, fallas de comunicación o configuración tardía del entorno.
  - **Paso 3**

    Analizar la causa raíz

    Se aplican técnicas como los “5 Porqués” o el Diagrama de Ishikawa para comprender el origen del problema.
  - **Paso 4**

    Definir aprendizajes

    Se documentan conclusiones que permitan evitar la repetición de errores metodológicos en futuras iteraciones o proyectos.

Las lecciones aprendidas carecen de impacto si se quedan archivadas en un documento estático. La socialización de este conocimiento es el mecanismo a través del

cual la experiencia individual se transforma en inteligencia colectiva. En el ámbito formativo, se debe motivar al aprendiz a compartir sus hallazgos, no desde la crítica destructiva, sino mediante talleres, reuniones de cierre o la actualización de bases de conocimiento internas del proyecto.

La socialización puede incluir la creación de guías de buenas prácticas derivadas de los errores cometidos. Si, por ejemplo, el equipo de pruebas descubre que las inyecciones SQL son vulnerabilidades frecuentes en el proyecto actual, la socialización puede consistir en capacitar brevemente a los desarrolladores sobre cómo parametrizar correctamente las consultas a la base de datos en los próximos ciclos de codificación.

Este flujo bidireccional de información fortalece la cohesión del equipo y eleva el estándar general de la producción de software.

- **Formas de socializar el conocimiento**
  - **Talleres internos**

    Permiten compartir hallazgos, explicar causas de errores y proponer buenas prácticas para próximos ciclos de trabajo.
  - **Reuniones de cierre**

    Facilitan la conversación del equipo sobre qué funcionó, qué falló y qué debe ajustarse en futuras iteraciones.
  - **Bases de conocimiento**

    Conservan aprendizajes, guías y recomendaciones para que otros integrantes del equipo puedan consultarlos posteriormente.
  - **Guías de buenas prácticas**

    Recogen orientaciones derivadas de errores reales, como vulnerabilidades frecuentes o fallas repetidas en el proceso de desarrollo.

4.2 Acciones correctivas, preventivas y estrategias de optimización

El punto central de la gestión de resultados es la formulación estructurada de planes de mejora. Todos los registros, métricas e incidentes recopilados previamente deben materializarse en acciones concretas, asignables y medibles que optimicen tanto el producto tecnológico como el proceso de construcción subyacente.

Un plan de mejora es la respuesta proactiva a las debilidades identificadas, asegurando que la calidad del software evolucione positivamente en el tiempo. En la ingeniería de calidad, las soluciones se dividen conceptualmente en dos grandes categorías: acciones correctivas y acciones preventivas.

Las acciones correctivas son reactivas; están diseñadas para solucionar un problema existente que ya ha impactado la calidad del software o del proceso. Por ejemplo, si las métricas revelan que los tiempos de ejecución de las pruebas manuales de regresión están retrasando las entregas, una acción correctiva sería reasignar recursos temporales o priorizar solo las pruebas de los módulos críticos.

Por otro lado, las acciones preventivas son proactivas; buscan eliminar la causa de un problema potencial antes de que ocurra. Siguiendo el mismo ejemplo, la acción preventiva a mediano plazo sería implementar un plan de capacitación en herramientas de automatización de pruebas para el equipo, de modo que en el futuro los ciclos de regresión se ejecuten de forma programática y rápida.

La habilidad para formular ambas tipologías de acciones demuestra una comprensión madura de la gestión de calidad, equilibrando la resolución de crisis inmediatas con la planificación estratégica.

- **Acciones correctivas y preventivas**
  - **Acciones correctivas**

    Son acciones reactivas orientadas a solucionar un problema existente que ya afectó la calidad del software o del proceso.
  - **Ejemplo correctivo**

    Si las pruebas manuales de regresión retrasan las entregas, se pueden reasignar recursos temporales o priorizar módulos críticos.
  - **Acciones preventivas**

    Son acciones proactivas orientadas a eliminar la causa de un problema potencial antes de que ocurra.
  - **Ejemplo preventivo**

    Si se prevén retrasos futuros en pruebas de regresión, se puede capacitar al equipo en herramientas de automatización de pruebas.
  - **Gestión de calidad**

    La combinación de acciones correctivas y preventivas permite atender problemas inmediatos y fortalecer la planificación estratégica.

La formulación de un plan de mejora no es un evento aislado, sino parte del ciclo iterativo de Planificar, Hacer, Verificar y Actuar, conocido como Ciclo de Deming o PDCA. Las estrategias de optimización continúan integradas en los planes de mejora deben ser medibles mediante indicadores de seguimiento.

Al establecer un plan, el profesional define claramente qué se va a hacer, quién es el responsable, qué recursos se requieren, en qué plazo debe completarse y cómo se medirá el éxito de la intervención.

En un contexto práctico de desarrollo web, si la retrospectiva identifica que la interfaz gráfica presenta múltiples defectos de responsivilidad en dispositivos móviles,

detectados mediante pruebas de caja negra, el plan de mejora establecería la obligación de incorporar simuladores móviles en las primeras fases del desarrollo front-end.

El éxito de esta estrategia de optimización se mediría comprobando, en el siguiente ciclo de pruebas, si la densidad de defectos relacionados con la interfaz móvil disminuyó significativamente. Así, el aprendiz comprende que el proceso de testing no es un obstáculo, sino el motor principal que impulsa la excelencia y la evolución técnica en la industria del software.

- **Ciclo de mejora continua aplicado a pruebas**
  - **Paso 1**

    Planificar

    Se define la acción de mejora, el responsable, los recursos requeridos, el plazo y el indicador de seguimiento.
  - **Paso 2**

    Hacer

    Se implementa la acción definida, como una capacitación, ajuste metodológico o incorporación de herramientas de apoyo.
  - **Paso 3**

    Verificar

    Se revisa si la acción produjo el efecto esperado, mediante indicadores como reducción de defectos o mejora en tiempos de prueba.
  - **Paso 4**

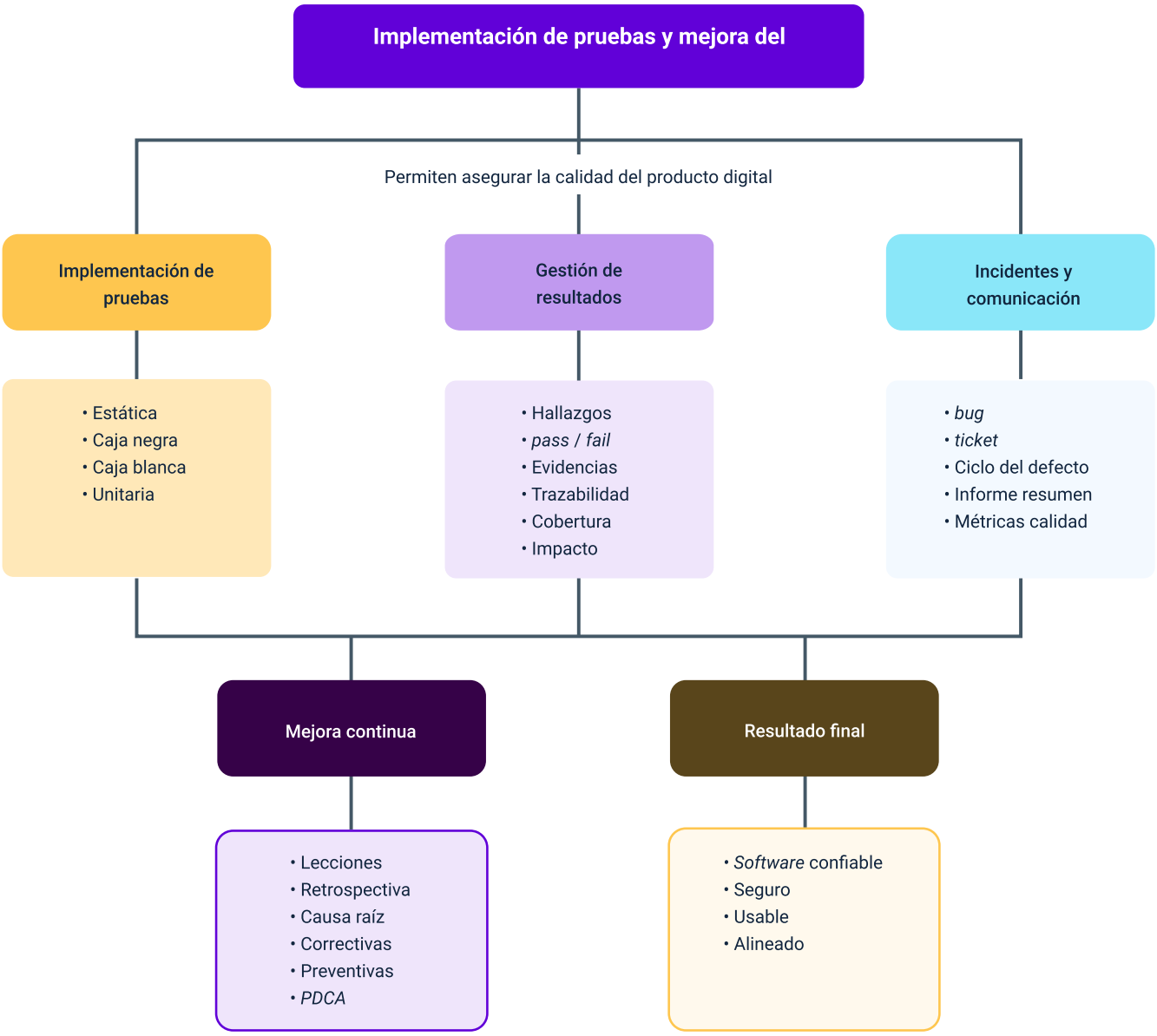
    Actuar

    Se institucionaliza la mejora cuando funciona o se ajusta la estrategia cuando los resultados no son suficientes.

Síntesis

La implementación de pruebas y mejora del software permite comprender que la calidad de una solución tecnológica no se asegura únicamente al final del desarrollo, sino durante todo el proceso. En este componente formativo se abordaron las pruebas estáticas, de caja negra, de caja blanca y unitarias como técnicas que permiten prevenir errores, validar funcionalidades, revisar la lógica interna del código y fortalecer la confiabilidad del producto. De esta manera, el aprendiz reconoce que probar un sistema implica aplicar procedimientos técnicos, registrar resultados y tomar decisiones con base en evidencias.

Asimismo, la gestión de resultados permite transformar los hallazgos del proceso de pruebas en información útil para la mejora continua. El registro de hallazgos, la trazabilidad, la estructuración de evidencias, la cobertura de pruebas, el reporte de incidentes, los informes de resumen, las métricas de calidad y las lecciones aprendidas aportan a la formulación de acciones correctivas, preventivas y estrategias de optimización. Así, el proceso de testing se convierte en una práctica técnica y analítica que contribuye a entregar soluciones de software más confiables, seguras, usables y alineadas con las necesidades del usuario y del proyecto.


Implementación de pruebas y mejora del
Permiten asegurar la calidad del producto digital
Implementación de pruebas
Gestión de resultados
Incidentes y comunicación
• Estática
• Caja negra
• Caja blanca
• Unitaria
• Hallazgos
• pass / fail
• Evidencias
• Trazabilidad
• Cobertura
• Impacto
• bug
• ticket
• Ciclo del defecto
• Informe resumen
• Métricas calidad
Mejora continua
Resultado final
• Lecciones
• Retrospectiva
• Causa raíz
• Correctivas
• Preventivas
• PDCA
• Software confiable
• Seguro
• Usable
• Alineado

Glosario

Aseguramiento de la calidad (QA, Quality Assurance): conjunto de actividades planificadas y sistemáticas implementadas en un sistema de calidad para garantizar que los procesos de desarrollo de software cumplan con los requisitos técnicos y estándares preestablecidos.

Automatización de pruebas: uso de software especializado y scripts para controlar la ejecución de pruebas y comparar los resultados reales con los esperados, reduciendo la intervención manual repetitiva.

Caso de prueba (test case): conjunto de condiciones, datos de entrada, acciones y resultados esperados, desarrollado para un objetivo particular, como ejercer una ruta de código específica o verificar el cumplimiento de un requisito determinado.

Ciclo de vida del defecto: trayectoria de estados por los que atraviesa el reporte de un error de software, generalmente abarcando las fases de descubrimiento, asignación, resolución, verificación y cierre.

Cobertura de código: métrica de análisis dinámico que mide, en porcentaje, qué partes del código fuente de un programa han sido ejecutadas durante la realización de un conjunto de casos de prueba.

Defecto (bug): imperfección o anomalía en un componente o sistema de software que puede causar que este no desempeñe su función requerida. se materializa cuando el código se ejecuta y produce un fallo.

Desarrollo guiado por pruebas (tdd): práctica de ingeniería de software en la que los desarrolladores escriben las pruebas unitarias automatizadas antes de escribir el código funcional que hará que dichas pruebas pasen.

Matriz de trazabilidad: documento estructurado en forma de tabla que correlaciona los requerimientos con los casos de prueba correspondientes, asegurando que cada necesidad del cliente sea validada.

Pruebas de caja blanca (white-box testing): método de prueba de software que examina las estructuras lógicas internas y el funcionamiento de una aplicación, en contraposición a su funcionalidad observable externamente.

Pruebas de caja negra (black-box testing): método de prueba basado en la especificación del sistema, donde el evaluador no tiene acceso ni conocimiento de la estructura del código fuente, por lo que evalúa exclusivamente las entradas y salidas de la interfaz.

Pruebas dinámicas: proceso de evaluación que requiere la compilación y ejecución real del código del software para analizar su comportamiento, uso de memoria y tiempos de respuesta bajo ciertas condiciones.

Pruebas estáticas: técnica de evaluación de artefactos de software, como documentos de requerimientos, modelos de diseño o código fuente, sin ejecutar la aplicación, enfocada en la detección temprana de anomalías.

Pruebas unitarias: nivel de prueba de software donde componentes individuales, como funciones, métodos o clases, se aíslan y prueban de manera independiente para verificar que su micrológica es correcta.

Regresión: aparición de un defecto en una parte del software que previamente funcionaba de manera correcta, usualmente causada por una modificación reciente en el código, una actualización o una corrección mal implementada.

Resultado esperado: comportamiento predecible y documentado que un sistema debe exhibir al ser sometido a un conjunto de datos de entrada o condiciones de ejecución específicas en un caso de prueba.

Resultado real: comportamiento o salida verificable que el sistema exhibe en el momento en que el evaluador ejecuta el caso de prueba, el cual se contrasta con el resultado esperado.

Retrospectiva: reunión formal realizada al finalizar un ciclo de trabajo o iteración, en la que el equipo evalúa su desempeño metodológico y define planes de acción para mejorar los procesos en futuros ciclos.

Referencias bibliográficas

Garousi, V., Giray, G., Tuzun, E., Catal, C., & Felderer, M. (2020). Closing the gap between software engineering education and industrial needs. IEEE Software, 37(2), 68–77.

International Organization for Standardization. (2022). ISO/IEC/IEEE 29119-1:2022 Software and systems engineering — Software testing — Part 1: General concepts ISO.

International Software Testing Qualifications Board. (2024). Certified tester foundation level syllabus (Version 4.0.1).

Jest Core Team. (s. f.). Jest.

JUnit Team. (s. f.). Junit user guide.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Sommerville, I. (2021). Engineering software products: An introduction to modern software engineering. Pearson.

Vera, M., & Silva, J. (2022). Estrategias pedagógicas para la enseñanza de calidad de software y pruebas automatizadas en entornos virtuales. Revista Iberoamericana de Tecnología en Educación y Educación en Tecnología, 31, 45–56.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Edison Eduardo Mantilla Cuadros	Responsable de línea de producción	Centro Agroturístico - Regional Santander
Carlos Andrés Bonza Reyes	Experto TIC	Centro Agroturístico – Regional Santander
Angelica Varón Quintero	Evaluadora instruccional	Centro Agroturístico - Regional Santander
Yuly Andrea Rey Quiñonez	Diseñadora de contenidos	Centro Agroturístico - Regional Santander
Lizeth Karina Manchego Suarez	Desarrolladora full stack	Centro Agroturístico - Regional Santander
María Alejandra Vera Briceño	Animadora y productora audiovisual	Centro Agroturístico - Regional Santander
Laura Paola Gelvez Manosalva	Validadora de recursos educativos digitales	Centro Agroturístico - Regional Santander
Sandra Liliana Cristancho Cruz	Evaluadora de contenidos inclusivos y accesibles	Centro Agroturístico - Regional Santander